

UNIT I LINEAR DATA STRUCTURES – LIST

Abstract Data Types (ADTs) – List ADT – array-based implementation – linked list implementation —singly linked lists- circularly linked lists- doubly-linked lists– applications of lists –Polynomial Manipulation – All operation (Insertion, Deletion, Merge, Traversal)

Data:

A collection of facts, concepts, figures, observations, occurrences or instructions in a formalized manner.

Information:

The meaning that is currently assigned to data by means of the conventions applied to those data(i.e. processed data)

Record:

Collection of related fields.

Data type:

Set of elements that share common set of properties used to solve a program.

Data Structures:

Data Structure is the way of organizing, storing, and retrieving data and their relationship with each other.

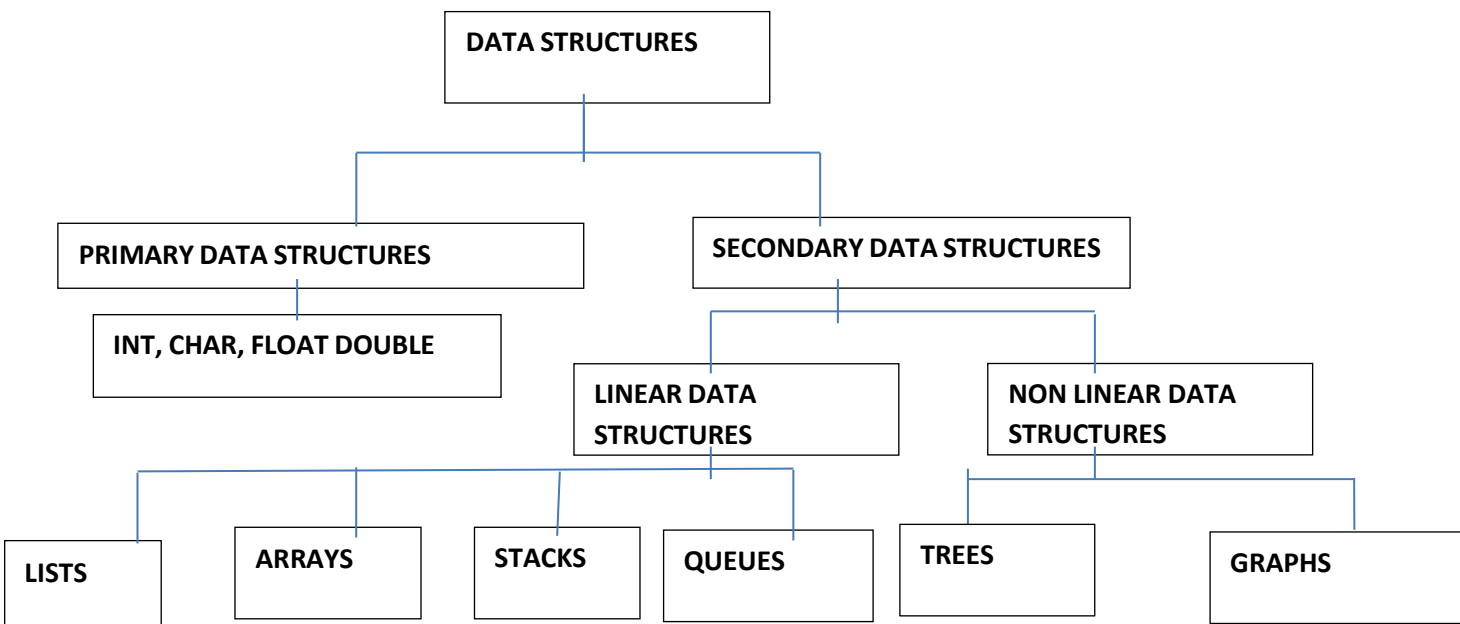
Characteristics of data structures:

1. It depicts the logical representation of data in computer memory.
2. It represents the logical relationship between the various data elements.
3. It helps in efficient manipulation of stored data elements.
4. It allows the programs to process the data in an efficient manner.

Operations on Data Structures:

- 1.Traversal
- 2.Search
- 3.Insertion
- 4.Deletion

CLASSIFICATION OF DATA STRUCTURES



Primary Data Structures/Primitive Data Structures:

Primitive data structures include all the fundamental data structures that can be directly

manipulated by machine-level instructions. Some of the common primitive data structures include integer, character, real, boolean etc

Secondary Data Structures/Non Primitive Data Structures:

Non primitive data structures, refer to all those data structures that are derived from one or more primitive data structures. The objective of creating non-primitive data structures is to form sets of homogeneous or heterogeneous data elements.

Linear Data Structures:

Linear data structures are data structures in which, all the data elements are arranged in a linear or sequential fashion. Examples of data structures include arrays, stacks, queues, linked lists, etc.

Non Linear Structures:

In non-linear data structures, there is definite order or sequence in which data elements are arranged. For instance, a non-linear data structure could arrange data elements in a hierarchical fashion. Examples of non-linear data structures are trees and graphs.

Static and dynamic data structure:

Static Ds:

If a ds is created using static memory allocation, i.e. ds formed when the number of data items are known in advance, it is known as static data structure or fixed size ds.

Dynamic Ds:

If the ds is created using dynamic memory allocation i.e ds formed when the number of data items are not known in advance is known as dynamic ds or variable size ds.

Application of data structures:

Data structures are widely applied in the following areas:

- Compiler design
- Operating system
- Statistical analysis package
- DBMS
- Numerical analysis
- Simulation
- Artificial intelligence
- Graphics

ABSTRACT DATA TYPES (ADTS):

An abstract Data type (ADT) is defined as a mathematical model with a collection of operations defined on that model. Set of integers, together with the operations of union, intersection and set difference form an example of an ADT. An ADT consists of data together with functions that operate on that data.

Advantages/Benefits of ADT:

- 1.Modularity
- 2.Reuse
- 3.code is easier to understand
- 4.Implementation of ADTs can be changed without requiring changes to the program that uses the ADTs.

THE LIST ADT:

List is an ordered set of elements.

The general form of the list is $A_1, A_2, \dots, A_N$

A_1 - First element of the list

A_2 - 1st element of the list

N –Size of the list

If the element at position i is A_i , then its successor is A_{i+1} and its predecessor is A_{i-1}

Various operations performed on List

1. Insert (X, 5)- Insert the element X after the position 5.
2. Delete (X) - The element X is deleted
3. Find (X) - Returns the position of X.
4. Next (i) - Returns the position of its successor element $i+1$.
5. Previous (i) Returns the position of its predecessor $i-1$.
6. Print list - Contents of the list is displayed.
7. Makeempty- Makes the list empty.

Implementation of list ADT:

1. Array based Implementation
2. Linked List based implementation

Array Implementation of list:

Array is a collection of specific number of same type of data stored in consecutive memory locations. Array is a static data structure i.e., the memory should be allocated in advance and the size is fixed. This will waste the memory space when used space is less than the allocated space.

Insertion and Deletion operation are expensive as it requires more data movements

Find and Print list operations takes constant time.

20	10	30	40	50	60
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]

The basic operations performed on a list of elements are

- a. Creation of List.
- b. Insertion of data in the List
- c. Deletion of data from the List
- d. Display all data"s in the List
- e. Searching for a data in the list

Declaration of Array:

```
#define maxsize 10  
int list[maxsize], n ;
```

Create Operation:

Create operation is used to create the list with „ n „, number of elements .If „ n „, exceeds the array's maxsize, then elements cannot be inserted into the list. Otherwise the array elements are stored in the consecutive array locations (i.e.) list [0], list [1] and so on.

```
void Create ( )
```

```
{  
    int i;  
    printf("\nEnter the number of elements to be added in the list:\t");  
    scanf("%d",&n);  
    printf("\nEnter the array elements:\t");  
    for(i=0;i<n;i++)  
        scanf("%d",&list[i]);  
}
```

If n=6, the output of creation is as follows:

list[6]

20	10	30	40	50	60
A[0]	A[1]	A[2]	A[3]	A[4]	A[5]

Insert Operation:

Insert operation is used to insert an element at particular position in the existing list. Inserting the element in the last position of an array is easy. But inserting the element at a particular position in an array is quite difficult since it involves all the subsequent elements to be shifted one position to the right.

Routine to insert an element in the array:

```
void Insert( )  
{  
    int i,data,pos;  
    printf("\nEnter the data to be inserted:\t");  
    scanf("%d",&data);  
    printf("\nEnter the position at which element to be inserted:\t");  
    scanf("%d",&pos);  
    if (pos==n)  
        printf("Array overflow");  
    for(i = n-1 ; i >= pos-1 ; i--)  
        list[i+1] = list[i];  
    list[pos-1] = data;  
    n=n+1;  
    Display();}
```

Consider an array with 5 elements [max elements = 10]

10	20	30	40	50					
----	----	----	----	----	--	--	--	--	--

If data 15 is to be inserted in the 2nd position then 50 has to be moved to next index position, 40 has to be moved to 50 position, 30 has to be moved to 40 position and 20 has to be moved to 30 position.

10	20	30	40	50					
----	----	----	----	----	--	--	--	--	--



10		20	30	40	50				
----	--	----	----	----	----	--	--	--	--

After this four data movement, 15 is inserted in the 2nd position of the array.

10	15	20	30	40	50				
----	----	----	----	----	----	--	--	--	--

Deletion Operation:

Deletion is the process of removing an element from the array at any position.

Deleting an element from the end is easy. If an element is to be deleted from any particular position, it requires all subsequent element from that position is shifted one position towards left.

Routine to delete an element in the array:

```
void Delete( )
{
    int i, pos ;
    printf("\nEnter the position of the data to be deleted:\t");
    scanf("%d",&pos);
    printf("\nThe data deleted is:\t %d", list[pos-1]);
    for(i=pos-1;i<n-1;i++)
        list[i]=list[i+1];
    n=n-1;
    Display();
}
```

Consider an array with 5 elements [max elements = 10]

10	20	30	40	50					
----	----	----	----	----	--	--	--	--	--

If data 20 is to be deleted from the array, then 30 has to be moved to data 20 position, 40 has to be moved to data 30 position and 50 has to be moved to data 40 position.

10	20	30	40	50					
----	----	----	----	----	--	--	--	--	--

After this 3 data movements, data 20 is deleted from the 2nd position of the array.

10	30	40	50						
----	----	----	----	--	--	--	--	--	--

Display Operation/Traversing a list

Traversal is the process of visiting the elements in a array.

Display() operation is used to display all the elements stored in the list. The elements are stored from the index 0 to n - 1. Using a for loop, the elements in the list are viewed

Routine to traverse/display elements of the array:

```
void display( )
{
    int i;
    printf("\n*****Elements in the array*****\n");
    for(i=0;i<n;i++)
        printf("%d\t",list[i]);
}
```

Search Operation:

Search() operation is used to determine whether a particular element is present in the list or not. Input the search element to be checked in the list.

Routine to search an element in the array:

```
void Search( )
{
    int search,i,count = 0;
    printf("\nEnter the element to be searched:\t");
    scanf("%d",&search);
    for(i=0;i<n;i++)
    {
        if(search == list[i])
            count++;
    }
}
```



```

                default: printf("\nEnter option between 1 - 6\n");
                        break;
        }
    }while(choice<7);
}
void Create()
{
    int i;
    printf("\nEnter the number of elements to be added in the list:\t");
    scanf("%d",&n);
    printf("\nEnter the array elements:\t");
    for(i=0;i<n;i++)
        scanf("%d",&list[i]);
    Display();
}
void Insert()
{
    int i,data,pos;
    printf("\nEnter the data to be inserted:\t");
    scanf("%d",&data);
    printf("\nEnter the position at which element to be inserted:\t");
    scanf("%d",&pos);
    for(i = n-1 ; i >= pos-1 ; i--)
        list[i+1] = list[i];
    list[pos-1] = data;
    n+=1;
    Display();
}
void Delete( )
{
    int i,pos;
    printf("\nEnter the position of the data to be deleted:\t");
    scanf("%d",&pos);
    printf("\nThe data deleted is:\t %d", list[pos-1]);
    for(i=pos-1;i<n-1;i++)
        list[i]=list[i+1];
    n=n-1;
    Display();
}
void Display()
{
    int i;
    printf("\n*****Elements in the array*****\n");
    for(i=0;i<n;i++)
        printf("%d\t",list[i]);
}

```

```

void Search()
{
    int search,i,count = 0;
    printf("\nEnter the element to be searched:\t");
    scanf("%d",&search);
    for(i=0;i<n;i++)
    {
        if(search == list[i])
        {
            count++;
        }
    }
    if(count==0)
        printf("\nElement not present in the list");
    else
        printf("\nElement present in the list");
}

```

Output

Array Implementation of List

- 1.create
- 2.Insert
- 3.Delete
- 4.Display
- 5.Search
- 6.Exit

Enter your choice: 1

Enter the number of elements to be added in the list: 5

Enter the array elements: 1 2 3 4 5

*****Elements in the array*****

1 2 3 4 5

Array Implementation of List

- 1.create
- 2.Insert
- 3.Delete
- 4.Display
- 5.Search
- 6.Exit

Enter your choice: 2

Enter the data to be inserted: 3

Enter the position at which element to be inserted: 1

*****Elements in the array*****

3 1 2 3 4 5

Array Implementation of List

- 1.create
- 2.Insert
- 3.Delete
- 4.Display
- 5.Search
- 6.Exit

Enter your choice: 3

Enter the position of the data to be deleted: 4

The data deleted is: 3

*****Elements in the array*****

3 1 2 4 5

Array Implementation of List

- 1.create
- 2.Insert
- 3.Delete
- 4.Display
- 5.Search
- 6.Exit

Enter your choice: 5

Enter the element to be searched: 1

Element present in the list

Array Implementation of List

- 1.create
- 2.Insert
- 3.Delete
- 4.Display
- 5.Search
- 6.Exit

Enter your choice:6

Advantages of array implementation:

- 1.The elements are faster to access using random access
- 2.Searching an element is easier

Limitation of array implementation

- An array store its nodes in consecutive memory locations.
- The number of elements in the array is fixed and it is not possible to change the number of elements .
- Insertion and deletion operation in array are expensive. Since insertion is performed by pushing the entire array one position down and deletion is performed by shifting the entire array one position up.

Applications of arrays:

Arrays are particularly used in programs that require storing large collection of similar type data elements.

Differences between Array based and Linked based implementation

	Array	Linked List
Definition	Array is a collection of elements having same data type with common name	Linked list is an ordered collection of elements which are connected by links/pointers
Access	Elements can be accessed using index/subscript, random access	Sequential access
Memory structure	Elements are stored in contiguous memory locations	Elements are stored at available memory space
Insertion & Deletion	Insertion and deletion takes more time in array	Insertion and deletion are fast and easy
Memory Allocation	Memory is allocated at compile time i.e static memory allocation	Memory is allocated at run time i.e dynamic memory allocation
Types	1D,2D,multi-dimensional	SLL, DLL circular linked list
Dependency	Each elements is independent	Each node is dependent on each other as address part contains address of next node in the list

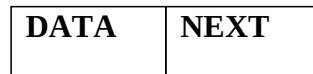
Linked list based implementation:

Linked Lists:

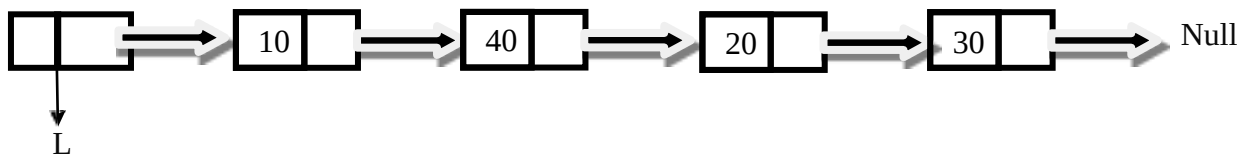
A Linked list is an ordered collection of elements. Each element in the list is referred as a node. Each node contains two fields namely,

Data field-The data field contains the actual data of the elements to be stored in the list

Next field- The next field contains the address of the next node in the list



A Linked list node



Advantages of Linked list:

1. Insertion and deletion of elements can be done efficiently
2. It uses dynamic memory allocation
3. Memory utilization is efficient compared to arrays

Disadvantages of linked list:

1. Linked list does not support random access
2. Memory is required to store next field
3. Searching takes time compared to arrays

Types of Linked List

1. Singly Linked List or One Way List
2. Doubly Linked List or Two-Way Linked List
3. Circular Linked List

Dynamic allocation

The process of allocating memory to the variables during execution of the program or at run time is known as dynamic memory allocation. C language has four library routines which allow this function.

Dynamic memory allocation gives best performance in situations in which we do not know memory requirements in advance. C provides four library routines to automatically allocate memory at the run time.

Memory allocation/de-allocation functions

Function	Task
<code>malloc()</code>	Allocates memory and returns a pointer to the first byte of allocated space
<code>calloc()</code>	Allocates space for an array of elements, initializes them to zero and returns a pointer to the memory
<code>free()</code>	Frees previously allocated memory
<code>realloc()</code>	Alters the size of previously allocated memory

To use dynamic memory allocation functions, you must include the header file `stdlib.h`.

malloc()

The `malloc` function reserves a block of memory of specified size and returns a pointer of type `void`. This means that we can assign it to any type of pointer.

The general syntax of `malloc()` is

```
ptr=(cast-type*)malloc(byte-size);
```

where `ptr` is a pointer of type `cast-type`. `malloc()` returns a pointer (of cast type) to an area of memory with size `byte-size`.

calloc():

`calloc()` function is another function that reserves memory at the run time. It is normally used to request multiple blocks of storage each of the same size and then sets all bytes to zero. `calloc()` stands for contiguous memory allocation and is primarily used to allocate memory for arrays.

The syntax of `calloc()` can be given as:

```
ptr=(cast-type*) calloc(n,elem-size);
```

The above statement allocates contiguous space for `n` blocks each of size `elem-size` bytes. The only difference between `malloc()` and `calloc()` is that when we use `calloc()`, all bytes are initialized to zero. `calloc()` returns a pointer to the first byte of the allocated region.

free():

The `free()` is used to release the block of memory.

Syntax:

The general syntax of the `free()` function is,

```
free(ptr);
```

where `ptr` is a pointer that has been created by using `malloc()` or `calloc()`. When memory is deallocated using `free()`, it is returned back to the free list within the heap.

realloc():

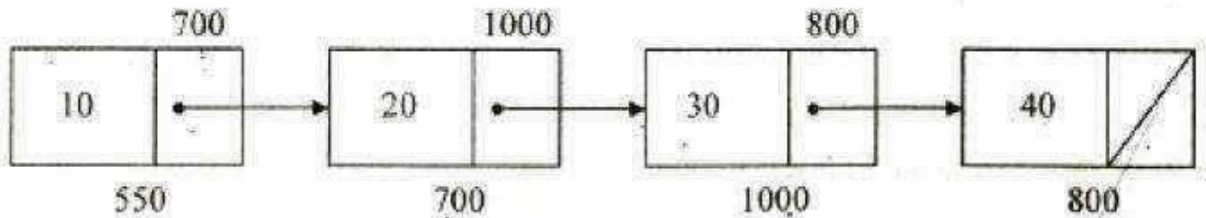
At times the memory allocated by using `calloc()` or `malloc()` might be insufficient or in excess. In both the situations we can always use `realloc()` to change the memory size already allocated by `calloc()` and `malloc()`. This process is called *reallocation of memory*. The general syntax for `realloc()` can be given as,

```
ptr = realloc(ptr,newsize);
```

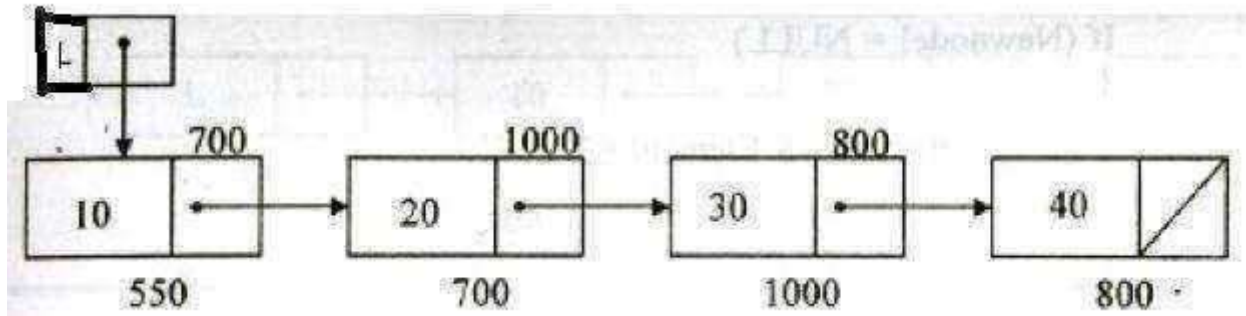
variable ptr. It returns a pointer to the first byte of the memory block. The allocated new block may be or may not be at the same region. Thus, we see that realloc() takes two arguments. The first is the pointer referencing the memory and the second is the total number of bytes you want to reallocate.

Singly Linked List

A singly linked list is a linked list in which each node contains only one link field pointing to the next node in the list



SLL



SLL with a Header

Basic operations on a singly-linked list are:

1. Insert – Inserts a new node in the list.
2. Delete – Deletes any node from the list.
3. Find – Finds the position(address) of any node in the list.
4. FindPrevious - Finds the position(address) of the previous node in the list.
5. FindNext- Finds the position(address) of the next node in the list.
6. Display-display the data in the list
7. Search-find whether a element is present in the list or not

Declaration of Linked List

```
void insert(int X,List L,position P);
void find(List L,int X); void
delete(int x , List L); typedef
struct node *position;
position L,p,newnode;
struct node
{
    int data;
    position next;
};
```

Creation of the list:

This routine creates a list by getting the number of nodes from the user. Assume n=4 for this example.

```
void create()
{
    int i,n;
    L=NULL;
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("\n Enter the number of nodes to be inserted\n");
    scanf("%d",&n);
    printf("\n Enter the data\n");
    scanf("%d",&newnode->data);
    newnode->next=NULL;
    L=newnode;
    p=L;
    for(i=2;i<=n;i++)
    {
        newnode=(struct node *)malloc(sizeof(struct node));
        scanf("%d",&newnode->data);
        newnode->next=NULL;
```

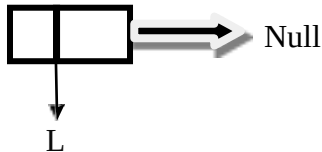
```

p->next=newnode;
p=newnode;
}
}

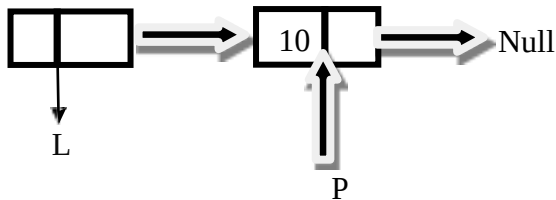
```

Initially the list is empty

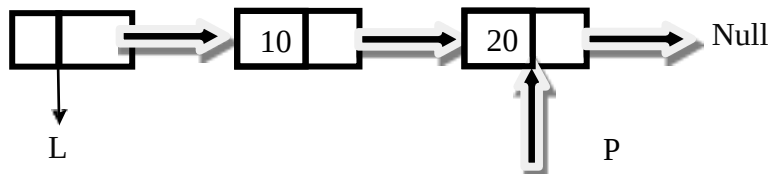
List L



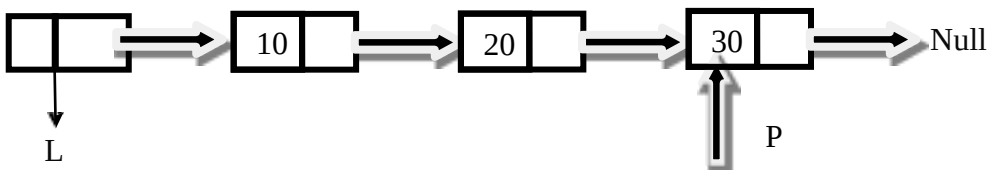
Insert(10,List L)- A new node with data 10 is inserted and the next field is updated to NULL. The next field of previous node is updated to store the address of new node.



Insert(20,L) - A new node with data 20 is inserted and the next field is updated to NULL. The next field of previous node is updated to store the address of new node.



Insert(30,L) - A new node with data 30 is inserted and the next field is updated to NULL. The next field of previous node is updated to store the address of new node.



Case 1:Routine to insert an element in list at the beginning

void insert(int X, List L, position p)

```
{  
  
    p=L;  
    newnode=(struct node*)malloc(sizeof(struct node));  
    printf("\nEnter the data to be  
    Inserted\n");scanf("%d",&newnode->data);  
    newnode->next=L; L=newnode;  
  
}
```

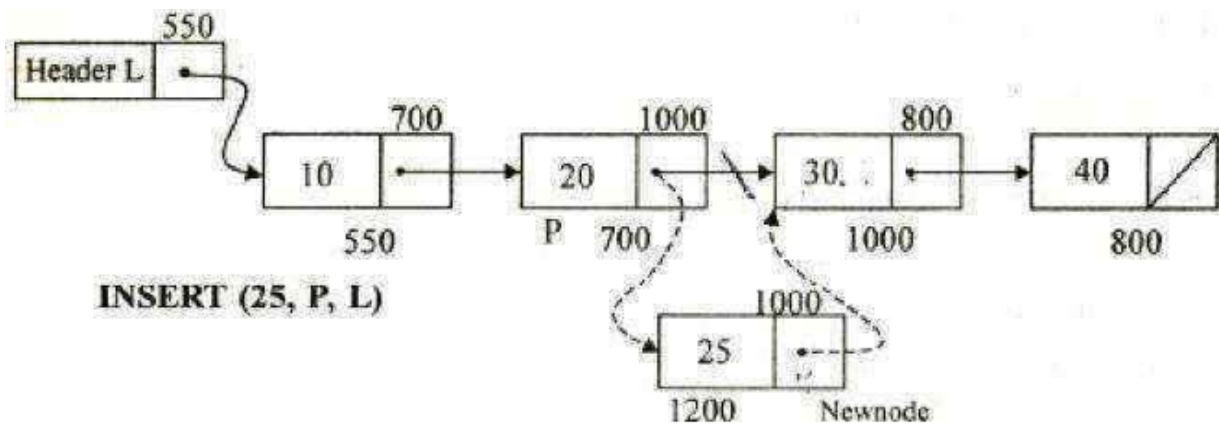
Case 2:Routine to insert an element in list at Position

This routine inserts an element X after the position P.

Void Insert(int X, List L, position p)

```
{  
    position newnode;  
    newnode =(struct node*) malloc( sizeof( struct node ));  
    if( newnode == NULL )  
        Fatal error( “ Out of Space ” );  
    else  
    {  
        Newnode -> data = x ;  
        Newnode -> next = p ->next ;  
        P -> next = newnode ;  
    }  
}
```

Insert(25,L, P) - A new node with data 25 is inserted after the position P and the next field is updated to NULL. The next field of previous node is updated to store the address of new node.



Case 3: Routine to insert an element in list at the end of the list

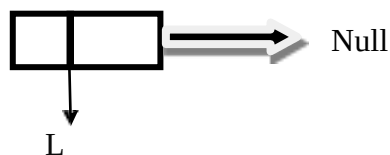
void insert(int X, List L, position p)

```
{
p=L;
    newnode=(struct node*)malloc(sizeof(struct node));
    printf("\nEnter the data to be inserted\n");
    scanf("%d",&newnode->data);
    while(p->next!=NULL)
    p=p->next;
    newnode->next=NULL;
    p->next=newnode;
    p=newnode;
}
```

Routine to check whether a list is Empty

This routine checks whether the list is empty .If the list is empty it returns 1

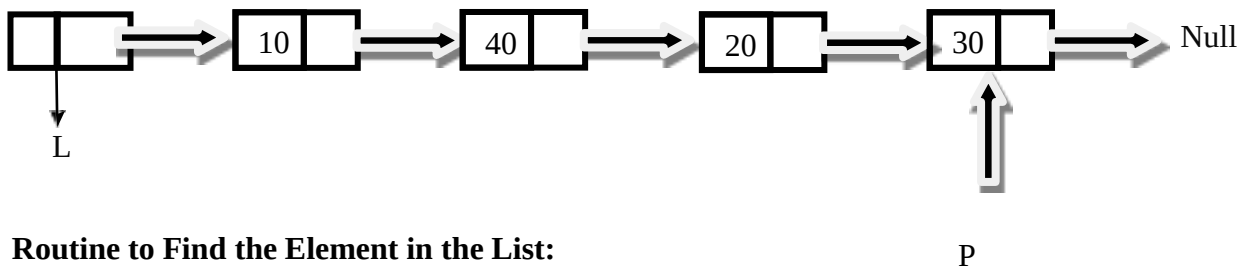
```
int IsEmpty( List L )
{
    if ( L -> next == NULL )
        return(1);
}
```



Routine to check whether the current position is last in the List

This routine checks whether the current position p is the last position in the list. It returns 1 if position p is the last position

```
int IsLast(List L , position p)
{
    if( p -> next == NULL)
        return(1);
}
```

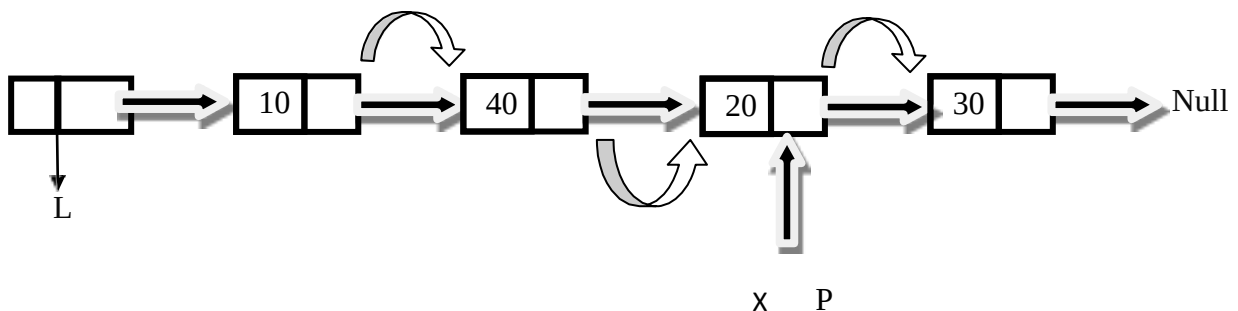


Routine to Find the Element in the List:

This routine returns the position of X in the list L

```
position find(List L, int X)
{
    position p;
    p=L->next;
    while(p!=NULL && p->data!=X)
        p=p->next;
    return(p);
}
```

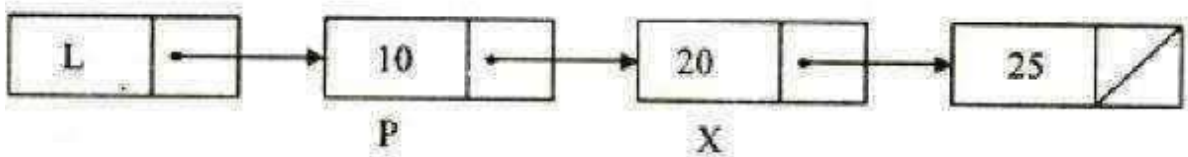
Find(List L , 20) - To find an element X traverse from the first node of the list and move to the next with the help of the address stored in the next field until data is equal to X or till the end of the list



Find Previous

It returns the position of its predecessor.

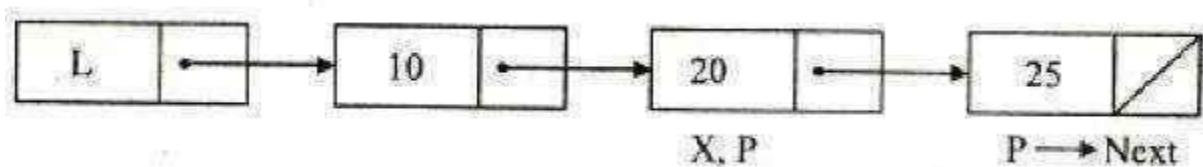
```
position FindPrevious (int X, List L)
{
    position p;
    p = L;
    while( p -> next != NULL && p -> next -> data != X )
        p = p -> next;
    return P;
}
```



Routine to find next Element in the List

It returns the position of successor.

```
void FindNext(int X, List L)
{
    position P;
    P=L->next;
    while(P!=NULL && P->data!=X)
        P = P->next;
    return P->next;
}
```



Routine to Count the Element in the List:

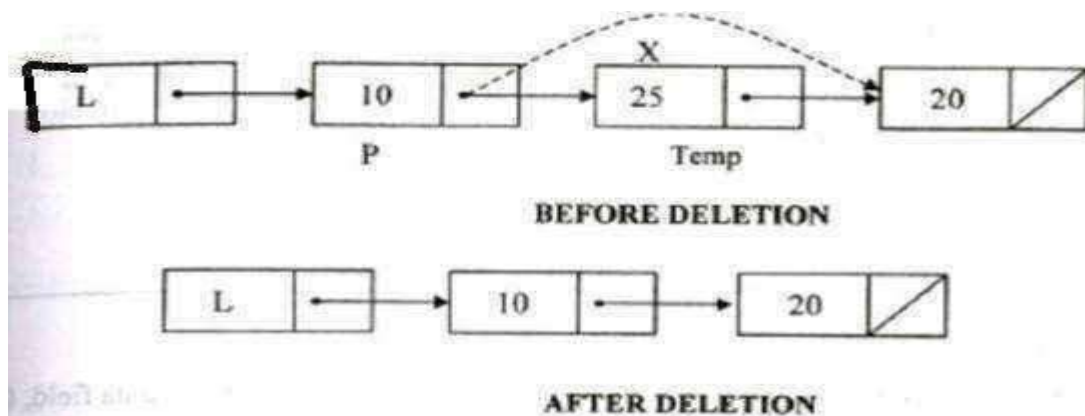
This routine counts the number of elements in the list

```
void count(List L)
{
    P = L -> next;
    while( p != NULL )
    {
        count++;
        p = p -> next;
    }
    print count;
}
```

Routine to Delete an Element in the List:

It delete the first occurrence of element X from the list L

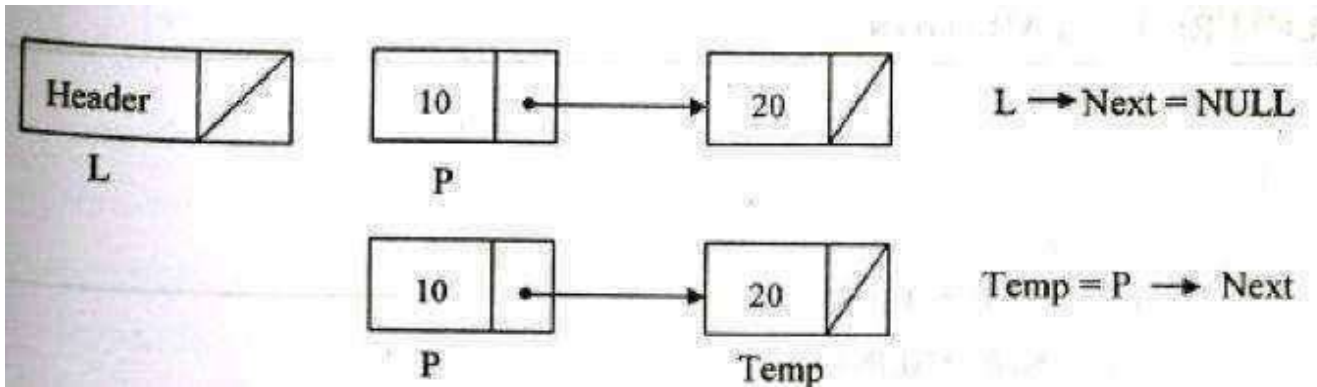
```
void Delete( int x , List L){
    position p, Temp;
    p = FindPrevious( X, L);
    if( ! IsLast (p, L)){
        temp = p -> next;
        P -> next = temp -> next;
        free ( temp );
    }
}
```



Routine to Delete the List

This routine deleted the entire list.

```
void Delete_list(List L)
{
    position P,temp;
    P=L->next;
    L->next=NULL;
    while(P!=NULL)
    {
        temp=P->next;
        free(P);
        P=temp;
    }
}
```



Program 1: Implementation of Singly linked List	Output
<pre>#include<stdio.h> #include<conio.h> #include<stdlib.h> void create(); void display(); void insert(); void find(); void delete(); typedef struct node *position; position L,p,newnode; struct node {</pre>	<pre>1.create 2.display 3.insert 4.find 5.delete Enter your choice</pre>

```

int data;
position next;
};
void main()
{
int choice;
clrscr();
do
{
printf("1.create\n2.display\n3.insert\n4.find\n5.delete\n\n\n");
printf("Enter your choice\n\n");
scanf("%d",&choice);
switch(choice)
{
case 1:
create();
break;
case 2:
display();
break;
case 3:
insert();
break;
case 4:
find();
break;
case 5:
delete();
break;
case 6:
exit(0);
}
}
while(choice<7);
getch();
}
void create()
{
int i,n;
L=NULL;
newnode=(struct node*)malloc(sizeof(struct node));
printf("\n Enter the number of nodes to be inserted\n");
scanf("%d",&n);
printf("\n Enter the data\n");
scanf("%d",&newnode->data);
newnode->next=NULL;

```

1

Enter the number of nodes to be inserted

5

Enter the data

1

2

3

4

5

1.create

2.display

3.insert

4.find

5.delete

Enter your choice

2

1 -> 2 -> 3 -> 4 -> 5 ->

Null

1.create

2.display

3.insert

4.find

5.delete

Enter your choice

3

```

L=newnode; p=L;
for(i=2;i<=n;i++)
{
newnode=(struct node *)malloc(sizeof(struct node));
scanf("%d",&newnode->data);
newnode->next=NULL;
p->next=newnode;
p=newnode;
}
}
void display()
{ p=L;
while(p!=NULL)
{
printf("%d -> ",p->data);
p=p->next;
}
printf("Null\n");
}
void insert()
{
int ch;

printf("\nEnter ur choice\n");
printf("\n1.first\n2.middle\n3.end\n");
scanf("%d",&ch);
switch(ch)
{
case 2:
{
int pos,i=1;
p=L;
newnode=(struct node*)malloc(sizeof(struct node));
printf("\nEnter the data to be inserted\n");
scanf("%d",&newnode->data);
printf("\nEnter the position to be inserted\n");
scanf("%d",&pos);
newnode->next=NULL;
while(i<pos-1)
{
p=p->next;
i++;
}
newnode->next=p->next;
p->next=newnode;
}
}
}

```

Enter ur choice

1.first
2.middle
3.end
1

Enter the data to be inserted

7
7 -> 1 -> 2 -> 3 -> 4 -> 5 -
> Null

1.create
2.display
3.insert
4.find
5.delete

Enter your choice

```

        p=newnode;
        display();
        break;
    }
case 1:
    {
        p=L;
        newnode=(struct node*)malloc(sizeof(struct node));
        printf("\nEnter the data to be inserted\n");
        scanf("%d",&newnode->data);
        newnode->next=L;
        L=newnode;
        display();
        break;
    }
case 3:
    {
        p=L;
        newnode=(struct node*)malloc(sizeof(struct node));
        printf("\nEnter the data to be inserted\n");
        scanf("%d",&newnode->data);
        while(p->next!=NULL)
            p=p->next;
        newnode->next=NULL;
        p->next=newnode;
        p=newnode;
        display();
        break;
    }
}
}
void find()
{
    int search,count=0;
    printf("\n Enter the element to be found:\n");
    scanf("%d",&search);
    p=L;
    while(p!=NULL)
    {
        if(p->data==search)
        {
            count++;
            break;
        }
        p=p->next;
    }
}

```

```

if(count==0)
printf("\n Element Not present\n");
else
printf("\n Element present in the list \n\n");
}
void delete()
{
position p,temp;
int x; p=L;
if(p==NULL)
{
printf("empty list\n");
}
else
{
printf("\nEnter the data to be deleted\n");
scanf("%d",&x);
if(x==p->data)
{ temp=p;
L=p->next;
free(temp);
display();
}
else
{
while(p->next!=NULL && p->next->data!=x)
{
p=p->next;
}
temp=p->next;
p->next=p->next->next;
free(temp);
display();
}
}
}

```

Advantages of SLL

1. The elements can be accessed using the next link

2. Occupies less memory than DLL as it has only one next field.

Disadvantages of SLL

1. Traversal in the backwards is not possible

2. Less efficient for insertion and deletion.

Doubly-Linked List

A doubly linked list is a linked list in which each node has three fields namely Data, Next, Prev.

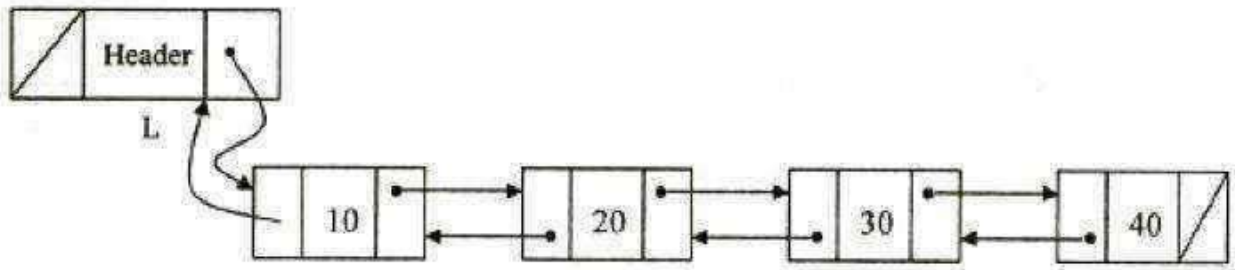
Data-This field stores the value of the element

Next-This field points to the successor node in the list

Prev-This field points to the predecessor node in the list

PREV	DATA	NEXT

DLL NODE



DOUBLY LINKED LIST

Basic operations of a doubly -linked list are:

1. Insert – Inserts a new element at the end of the list.
2. Delete – Deletes any node from the list.
3. Find – Finds any node in the list.
4. Print – Prints the list

Declaration of DLL Node

```
typedef struct node *position ;
```

```
struct node
```

```
{
```

```
int data;
```

```
    position prev;
```

```
    position next;
```

```
};
```

PREV	DATA	NEXT

Creation of list in DLL

Initially the list is empty. Then assign the first node as head.

```
newnode->data=X;  
newnode->next=NULL;  
newnode->prev=NULL;  
L=newnode;
```

list. If we add one more node in the list, then create a newnode and attach that node to the end of the

```
L->next=newnode;  
newnode->prev=L;
```

Routine to insert an element in a DLL at the beginning

```
void Insert (int x, list L, position P){  
    struct node *Newnode;  
  
    if(pos==1)  
        P=L;  
  
    Newnode = (struct node*)malloc (sizeof(struct node));  
  
    if (Newnode! = NULL)  
        Newnode->data= X;  
  
    Newnode ->next= L ->next;  
  
    L->next ->prev=Newnode  
  
    L->next = Newnode;  
  
    Newnode ->prev = L;  
  
}
```

Routine to insert an element in a DLL any position :

```
void Insert (int x, list L, position P)
{
    struct node *Newnode;

    Newnode = (struct node*)malloc (sizeof(struct node));

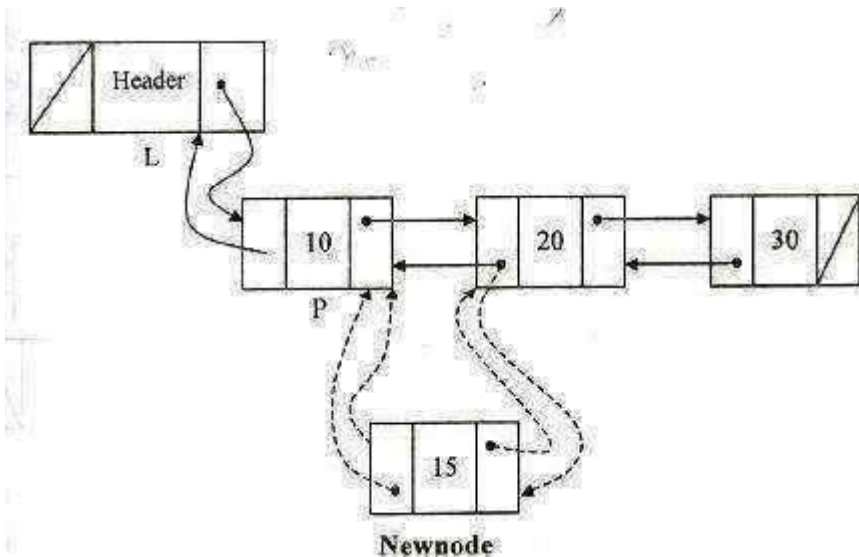
    if (Newnode != NULL)
    {
        Newnode->data= x;

        Newnode ->next= P ->next;

        P->next ->prev=Newnode

        P ->next = Newnode;

        Newnode ->prev = P;
    }
}
```



Routine to insert an element in a DLL at the end:

```
void insert(int X, List L, position p)
{
    p=L;

    newnode=(struct node*)malloc(sizeof(struct node));
```

```

printf("\nEnter the data to be inserted\n");
scanf("%d",&newnode->data);
while(p->next!=NULL)
p=p->next;
newnode->next=NULL;
p->next=newnode;
newnode->prev=p;
}

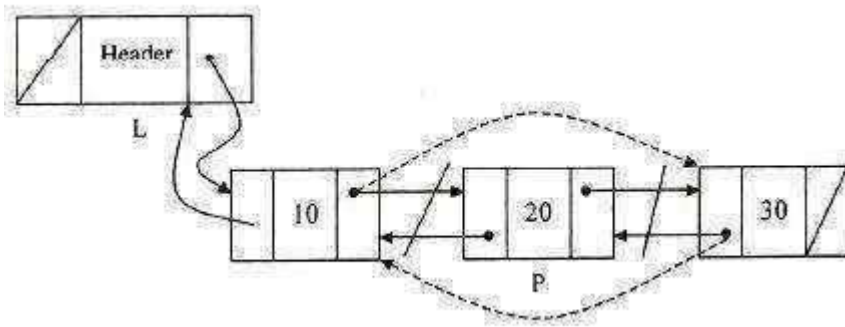
```

Routine for deleting an element:

```

void Delete (int x ,List L)
{
    Position p , temp;
    P = Find( x, L );
    if(P==L->next)
    temp=L;
    L->next=temp->next;
    temp->next->prev=L;
    free(temp);
    elseif( IsLast( p, L ) )
    {
        temp = p;
        p -> prev -> next = NULL;
        free(temp);
    }
    else
    {
        temp = p;
        p -> prev -> next = p -> next;
        p -> next -> prev = p -> prev;
        free(temp);
    }
}

```



Routine to display the elements in the list:

```

void Display( List L )
{
    P = L -> next ;
    while ( p != NULL)
    {
        printf("%d", p -> data ;
        p = p -> next ;
    }
    printf("NULL");
}

```

Routine to search whether an element is present in the list

```

void find()
{
    int a,flag=0,count=0;
    if(L==NULL)
        printf("\nThe list is empty");
    else
    {
        printf("\nEnter the elements to be searched");
        scanf("%d",&a);
        for(P=L;P!=NULL;P=P->next)
        {
            count++;
            if(P->data==a)
            {
                flag=1;
                printf("\nThe element is found");
                printf("\nThe position is %d",count);
            }
        }
    }
}

```

```

break;
}
}
if(flag==0)
printf("\nThe element is not found");
}
}

```

Program 2-Implementation of Doubly linked list	Output
<pre> #include<conio.h> void insert(); void deletion(); void display(); void find(); typedef struct node *position; position newnode,temp,L=NULL,P; struct node { int data; position next; position prev; }; void main() { int choice; clrscr(); do { printf("\n1.INSERT"); printf("\n2.DELETE"); printf("\n3.DISPLAY"); printf("\n4.FIND"); printf("\n5.EXIT"); printf("\nEnter ur option"); scanf("%d",&choice); switch(choice) { case 1: insert(); break; case 2: deletion(); break; case 3: display(); </pre>	<pre> 1.INSERT 2.DELETE 3.DISPLAY 4.FIND 5.EXIT Enter ur option1 Enter the data to be inserted10 1.INSERT 2.DELETE 3.DISPLAY 4.FIND 5.EXIT Enter ur option1 Enter the data to be inserted 20 Enter the position where the data is to be inserted 2 </pre>

```

break;
case 4:
find();
break;
case 5:
exit(1);
}
}while(choice!=5);
getch();
}
void insert()
{
int pos,I;
newnode=(struct node*)malloc(sizeof(struct node));
printf("\nEnter the data to be inserted");
scanf("%d",&newnode->data);
if(L==NULL)
{
L=newnode;
L->next=NULL;
L->prev=NULL;
}
else
{
printf("\nEnter the position where the data is to be inserted");
scanf("%d",&pos);
if(pos==1)
{
newnode->next=L;
newnode->prev=NULL;
L->prev=newnode;
L=newnode;
}
else
{
P=L;
for(i=1;i<pos-1&&P->next!=NULL;i++)
{
P=P->next;
}
newnode->next=P->next;
P->next=newnode;
newnode->prev=P;
P->next->prev=newnode;
}
}
}

```

- 1.INSERT
- 2.DELETE
- 3.DISPLAY
- 4.FIND
- 5.EXIT

Enter ur option1

Enter the data to be inserted 30

Enter the position where the data is to be inserted3

- 1.INSERT
- 2.DELETE
- 3.DISPLAY
- 4.FIND
- 5.EXIT

Enter ur option 3

The elements in the list are

10 20 30

- 1.INSERT
- 2.DELETE
- 3.DISPLAY
- 4.FIND
- 5.EXIT

Enter ur option 2

}

<pre> } void deletion() { int pos,i; if(L==NULL) printf("\nThe list is empty"); else { printf("\nEnter the position of the data to be deleted"); scanf("%d",&pos); if(pos==1) { temp=L; L=temp->next; L->prev=NULL; printf("\nThe deleted element is %d",temp->data); free(temp); } else { P=L; for(i=1;i<pos-1;i++) P=P->next; temp=P->next; printf("\nThe deleted element is %d",temp->data); P->next=temp->next; temp->next->prev=P; free(temp); } } } void display() { if(L==NULL) printf("\nNo of elements in the list"); else { printf("\nThe elements in the listare\n"); for(P=L;P!=NULL;P=P->next) printf("%d",P->data); } } void find() { int a,flag=0,count=0; </pre>	Enter the position of the data to be deleted 2 The deleted element is 20 1.INSERT 2.DELETE 3.DISPLAY 4.FIND 5.EXIT Enter ur option 3 The elements in the list are 10 30 1.INSERT 2.DELETE 3.DISPLAY 4.FIND 5.EXIT Enter ur option4 Enter the elements to be searched 20 The element is not found 1.INSERT 2.DELETE 3.DISPLAY 4.FIND 5.EXIT Enter ur option 4
--	--

```

if(L==NULL)
printf("\nThe list is empty");
else
{
printf("\nEnter the elements to be searched");
scanf("%d",&a);
for(P=L;P!=NULL;P=P->next)
{
count++;
if(P->data==a)
{
flag=1;
printf("\nThe element is found");
printf("\nThe position is %d",count);
break;
}
}
if(flag==0)
printf("\nThe element is not found");
}
}

```

Enter the elements to be searched 30

The element is found

The position is 2

1.INSERT

2.DELETE

3.DISPLAY

4.FIND

5.EXIT

Enter ur option5

Press any key to continue .

..

Advantages of DLL:

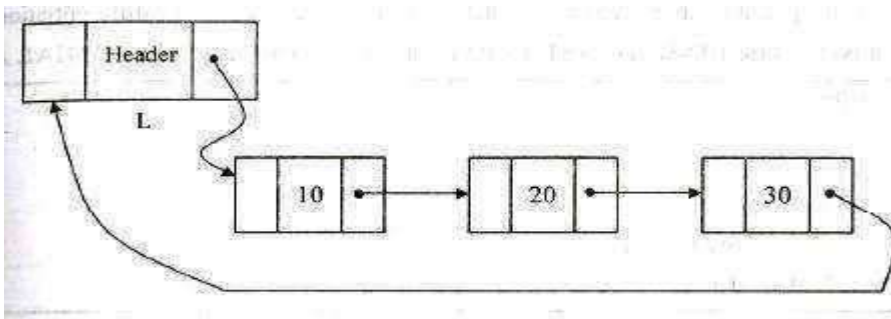
The DLL has two pointer fields. One field is prev link field and another is next link field. Because of these two pointer fields we can access any node efficiently whereas in SLL only one link field is there which stores next node which makes accessing of any node difficult.

Disadvantages of DLL:

The DLL has two pointer fields. One field is prev link field and another is next link field. Because of these two pointer fields, more memory space is used by DLL compared to SLL

CIRCULAR LINKED LIST:

Circular Linked list is a linked list in which the pointer of the last node points to the first node.

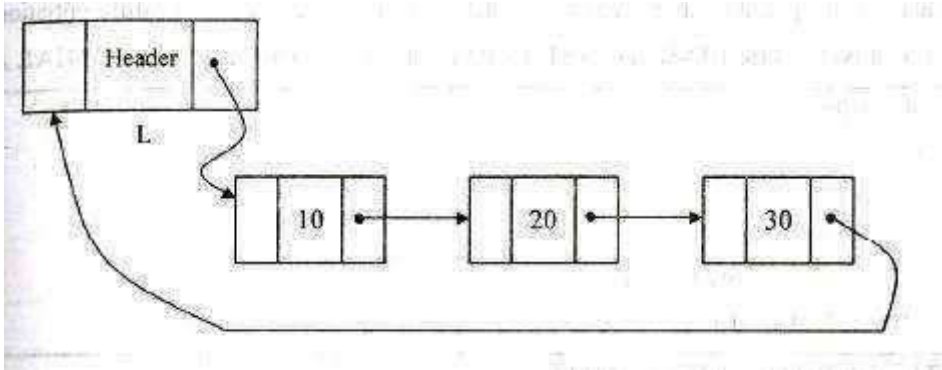


Types of CLL:

CLL can be implemented as circular singly linked list and circular doubly linked list.

Singly linked circular list:

A Singly linked circular list is a linked list in which the last node of the list points to the first node.



Declaration of node:

```
typedef struct node *position;
```

```
struct node
```

```
{
```

```
int data;
```

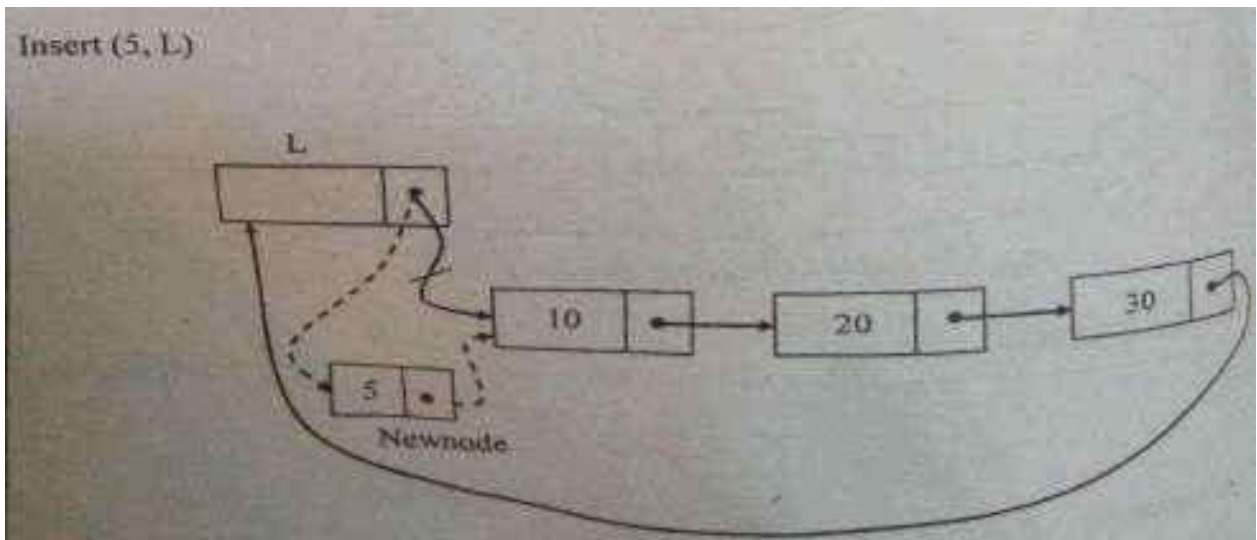
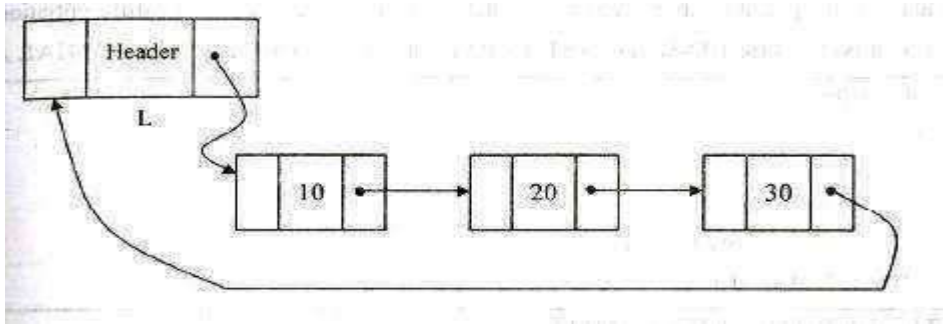
```
position next;
```

```
};
```

Routine to insert an element in the beginning

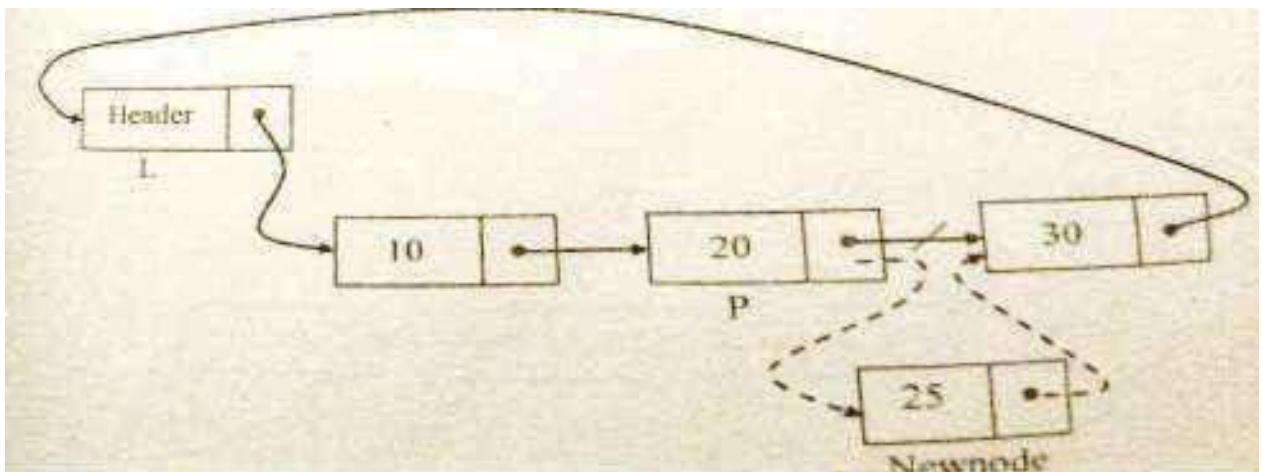
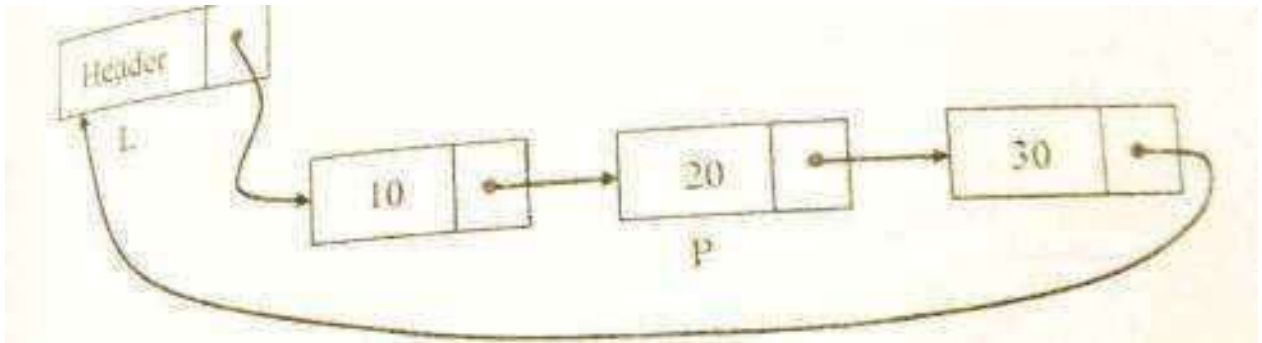
```
void insert_beg(int X,List L)
```

```
{  
    position Newnode;  
    Newnode=(struct node*)malloc(sizeof(struct node));  
    if(Newnode!=NULL)  
    {  
        Newnode->data=X;  
        Newnode->next=L->next;  
        L->next=Newnode;  
    }  
}
```



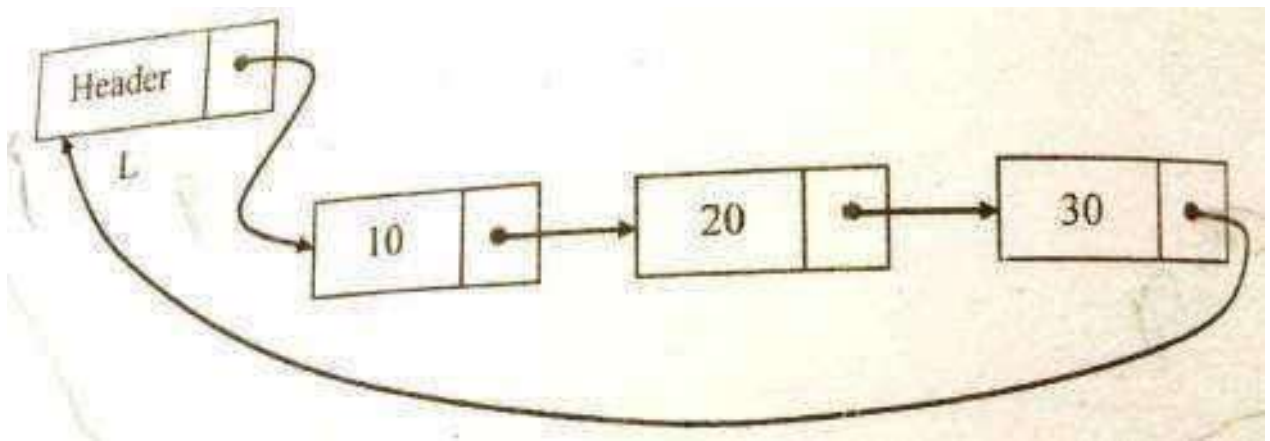
Routine to insert an element in the middle

```
void insert_mid(int X, List L, Position p)
{
    position Newnode;
    Newnode=(struct node*)malloc(sizeof(struct node));
    if(Newnode!=NULL)
    {
        Newnode->data=X;
        Newnode->next=p->next;
        p->next=Newnode;
    }
}
```

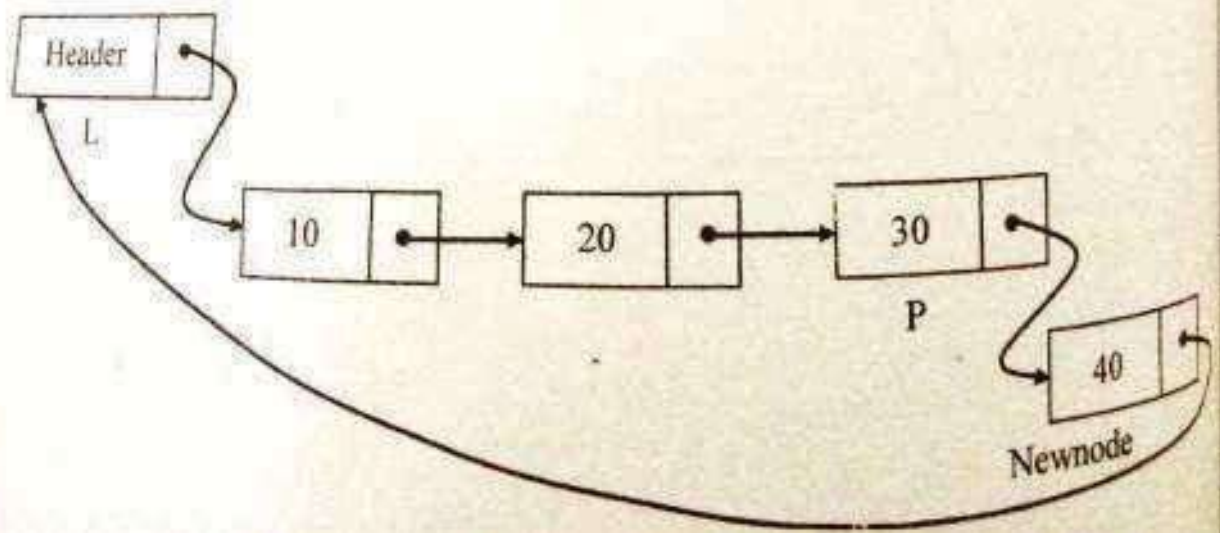


Routine to insert an element in the last

```
void insert_last(int X, List L)
position Newnode;
Newnode=(struct node*)malloc(sizeof(struct node));
if(Newnode!=NULL)
{
P=L;
while(P->next!=L)
P=P->next;
Newnode->data=X;
P->next=Newnode;
Newnode->next=L;
}
}
```

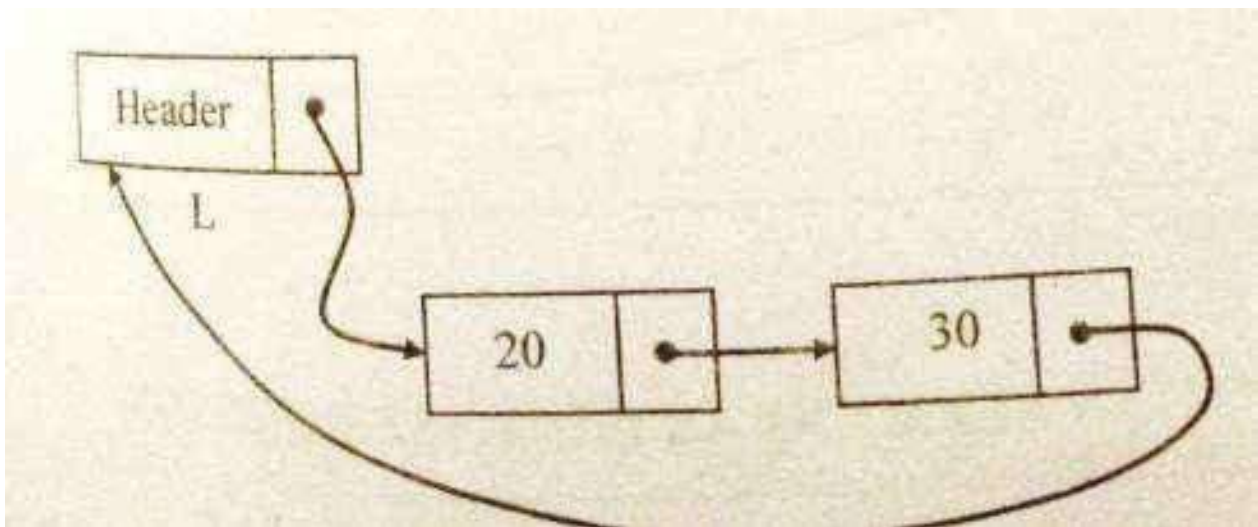
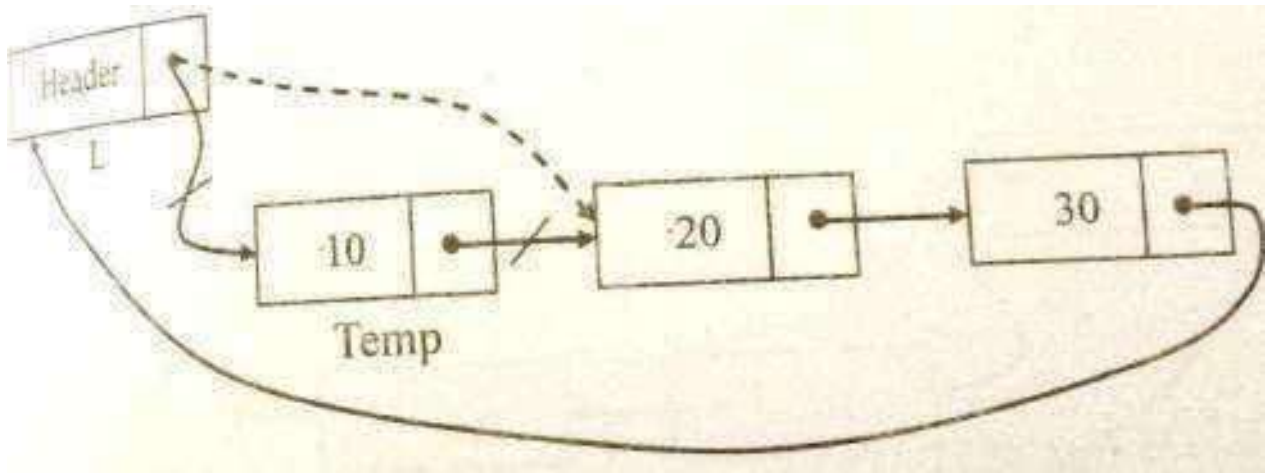


insert (40, L)



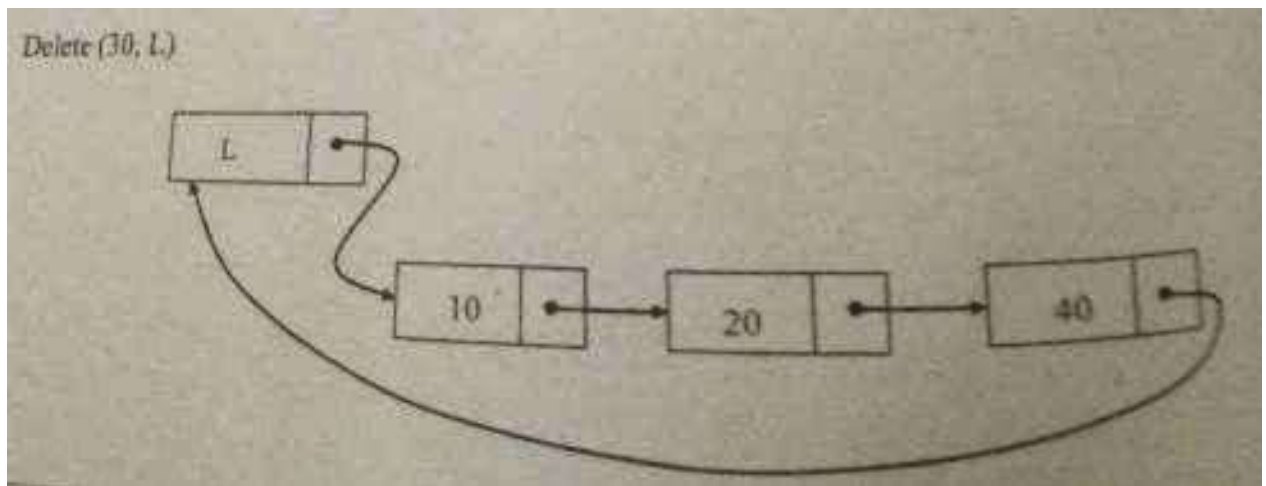
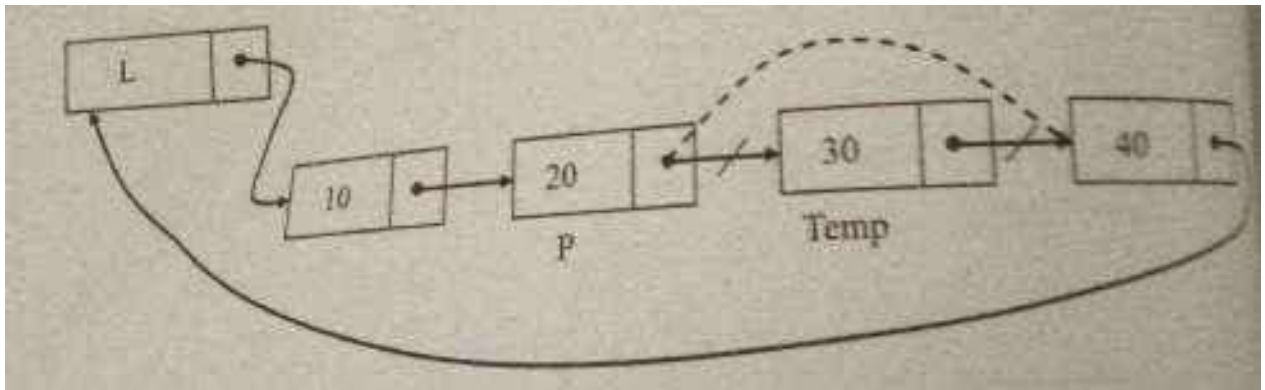
Routine to delete an element from the beginning

```
void del_first(List L)
{
    position temp;
    temp=L->next;
    L->next=temp->next;
    free(temp);
}
```



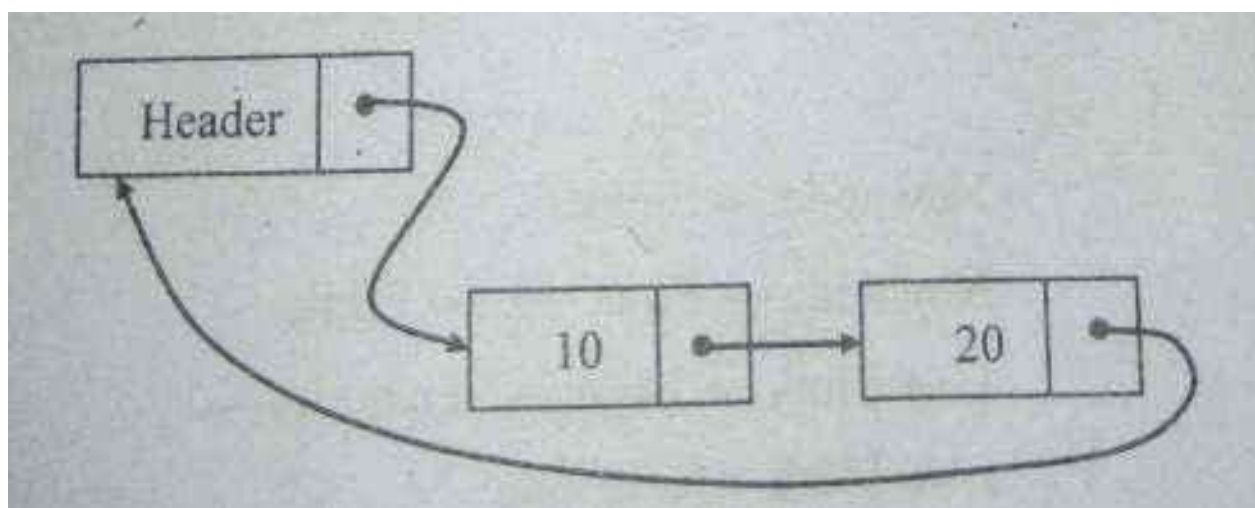
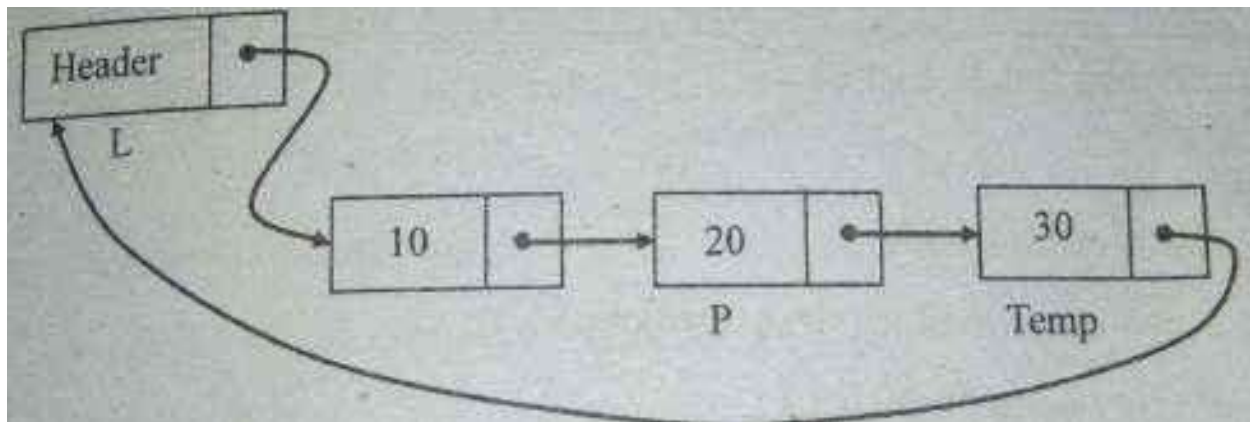
Routine to delete an element from the middle

```
void del_mid(int X,List L)
{
    position p, temp;
    p=findprevious(X,L);
    if(!IsLast(P,L))
    {
        temp=p->next;
        p->next=temp->next;
        free(temp);
    }
}
```



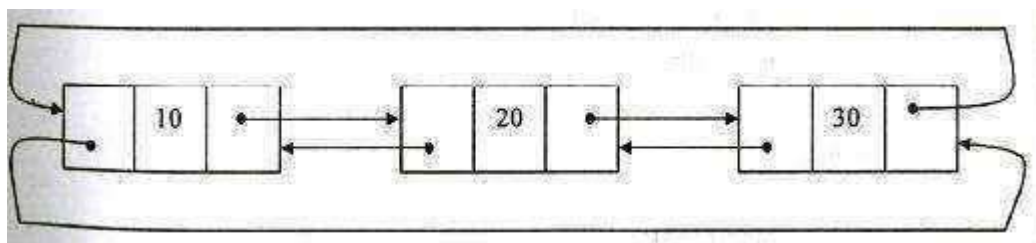
Routine to delete an element at the last position

```
void del_last(List L)
{
    position p, temp;
    p=L;
    while(p->next->next!=L)
    p=p->next;
    temp=p->next;
    p->next=L
    free(temp);}
```



Doubly Linked circular list:

A doubly linked circular list is a doubly linked list in which the next link of the last node points to the first node and prev link of the first node points to the last node of the list.

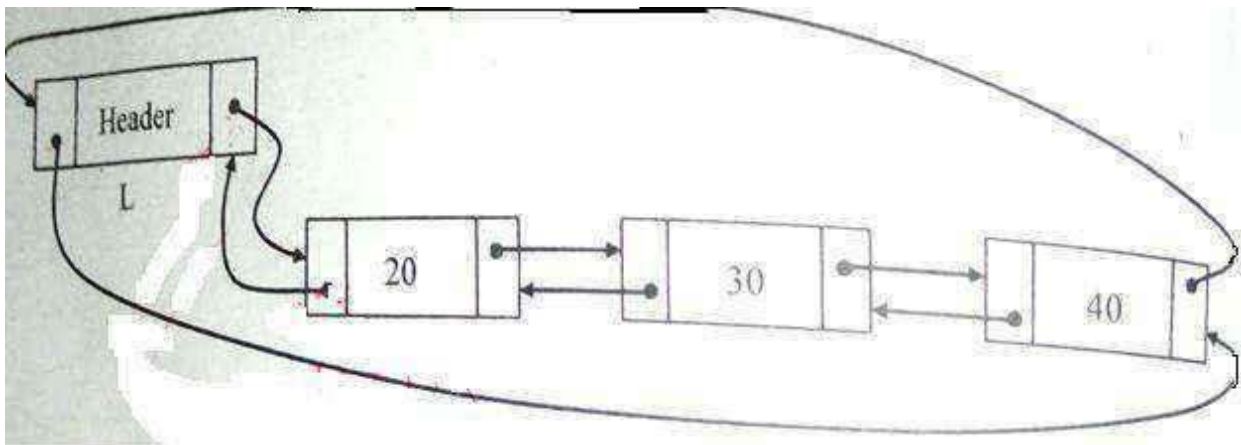


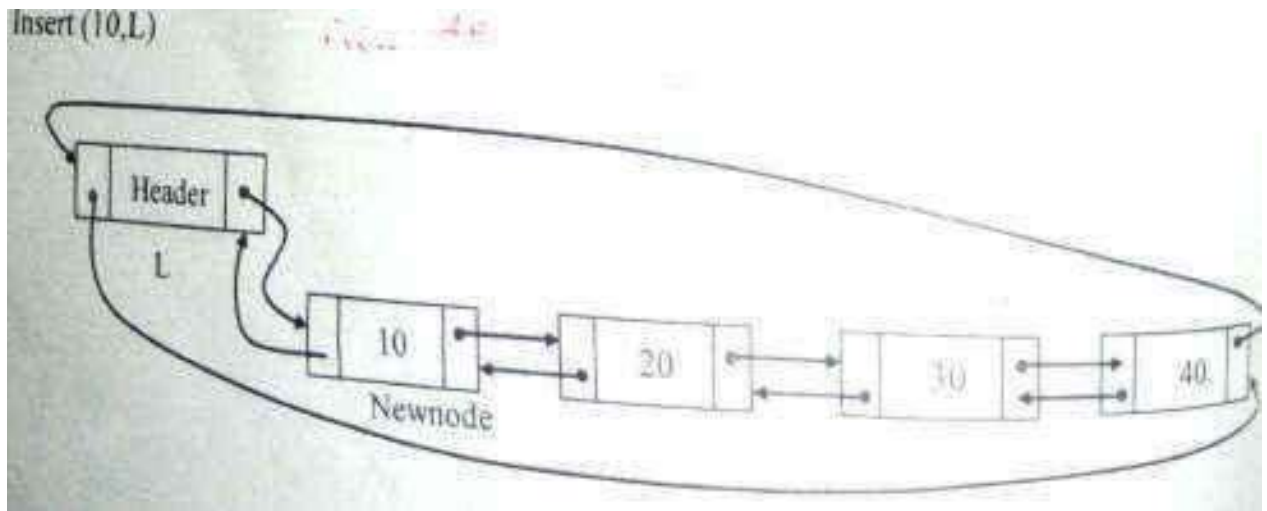
Declaration of node:

```
typedef struct node *position;  
struct node  
{  
    int data;  
    position next;  
    position prev;  
};
```

Routine to insert an element in the beginning

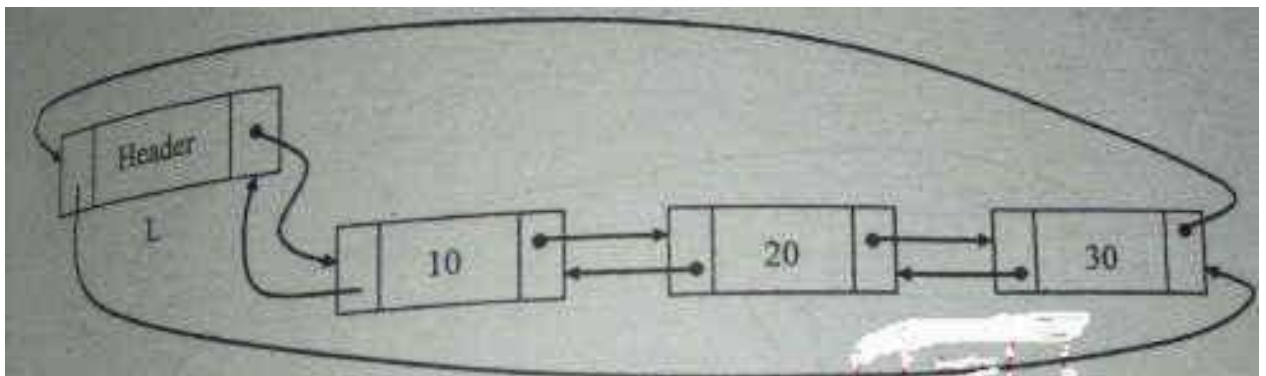
```
void insert_beg(int X,List L)  
{  
    position Newnode;  
    Newnode=(struct node*)malloc(sizeof(struct node));  
    if(Newnode!=NULL)  
    {  
        Newnode->data=X;  
        Newnode->next=L->next;  
        L->next->prev=Newnode;  
        L->next=Newnode;  
        Newnode->prev=L;  
    }  
}
```

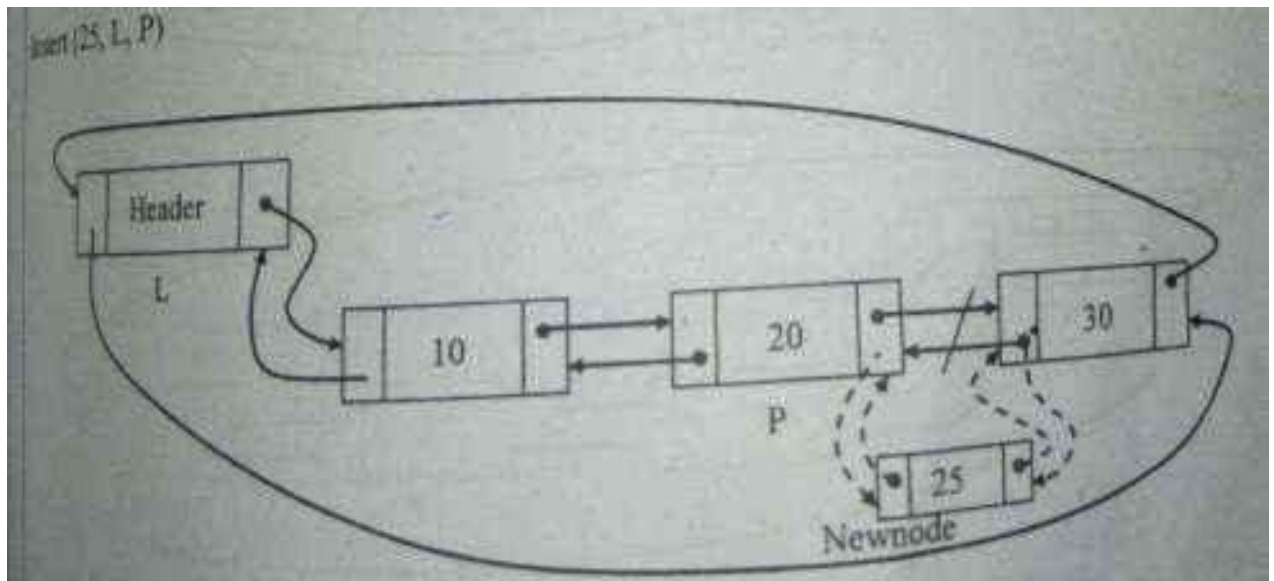




Routine to insert an element in the middle

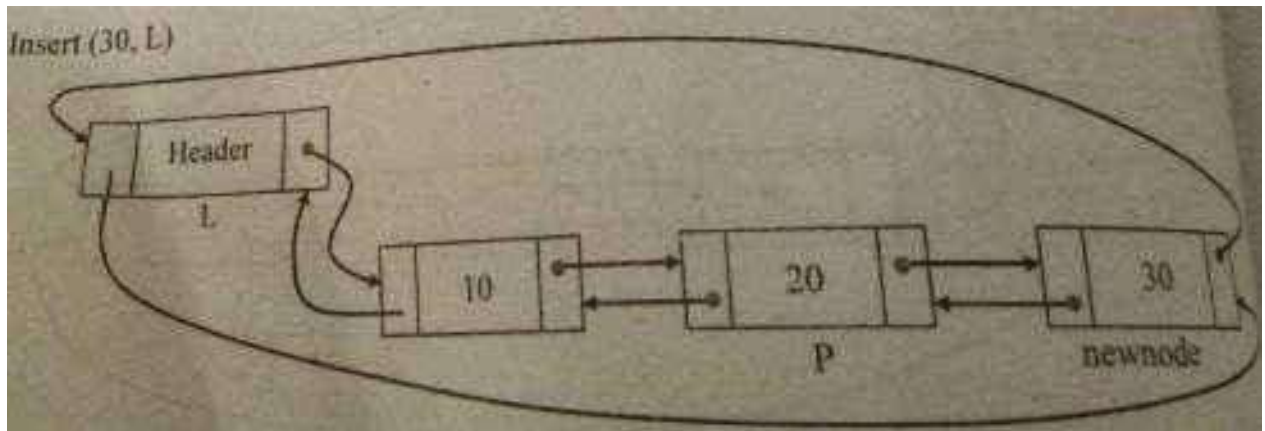
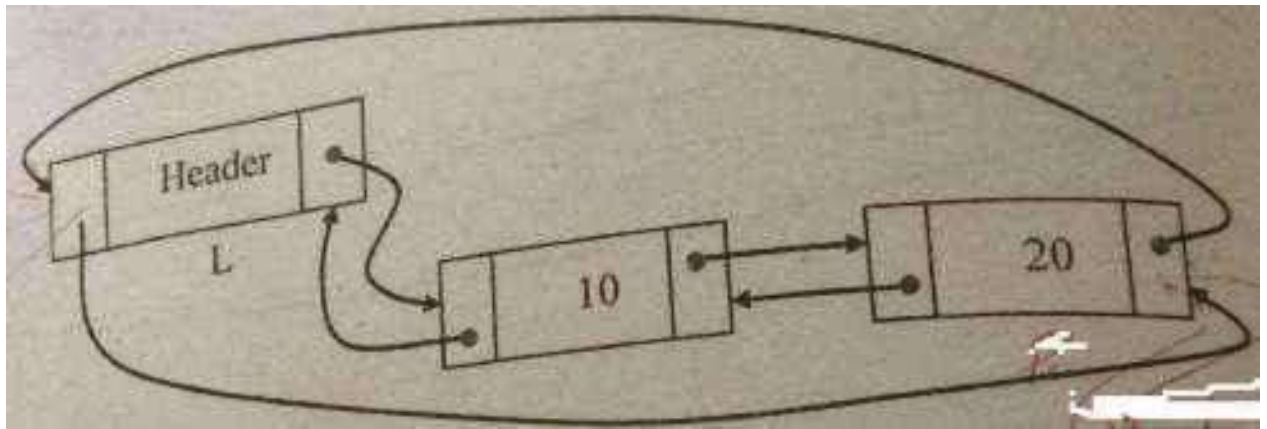
```
void insert_mid(int X, List L, Position p)
{
    position Newnode;
    Newnode=(struct node*)malloc(sizeof(struct node));
    if(Newnode!=NULL)
    {
        Newnode->data=X;
        Newnode->next=p->next;
        p->next->prev=Newnode;
        p->next=Newnode;
        Newnode->prev=p;
    }
}
```





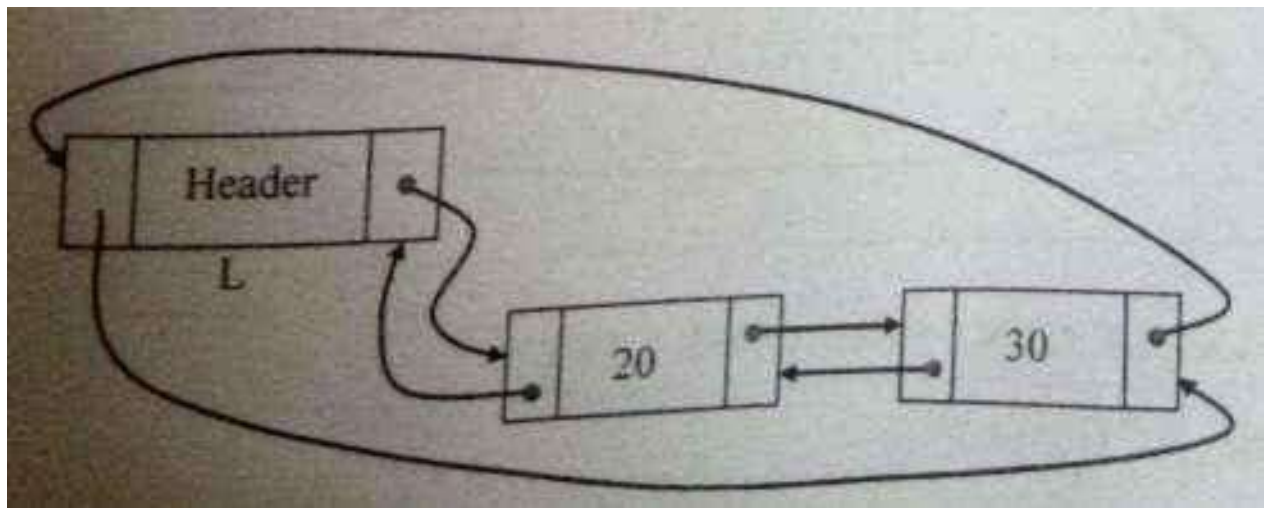
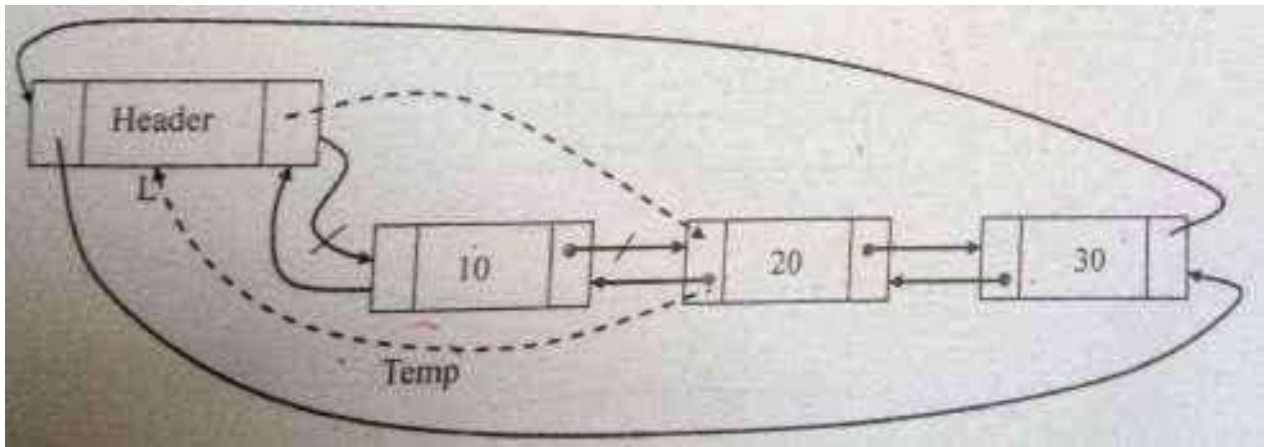
Routine to insert an element in the last

```
void insert_last(int X, List L)
{
    position Newnode,p;
    Newnode=(struct node*)malloc(sizeof(struct node));
    if(Newnode!=NULL)
    {
        p=L;
        while(p->next!=L)
        p=p->next;
        Newnode->data=X;
        p->next =Newnode;
        Newnode->next=L;
        Newnode->prev=p;
        L->prev=newnode;
    }
}
```



Routine to delete an element from the beginning

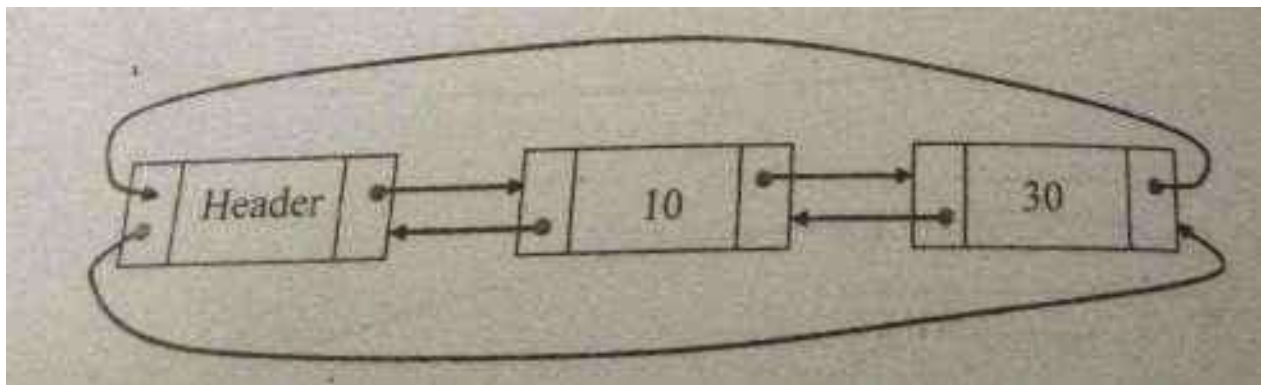
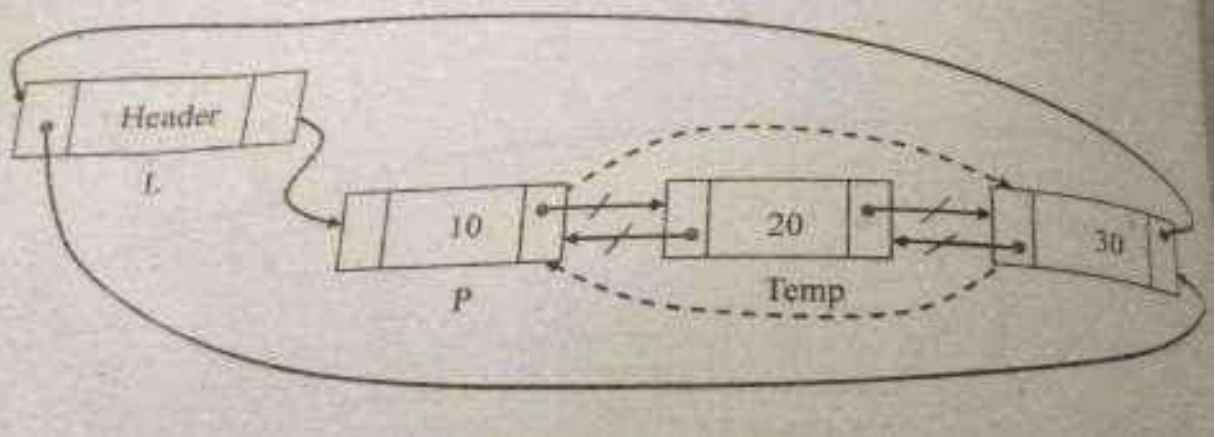
```
void del_first(List L)
{
    position temp;
    if(L->next!=NULL)
    {
        temp=L->next;
        L->next=temp->next;
        temp->next->prev=L;
        free(temp);
    }
}
```



Routine to delete an element from the middle

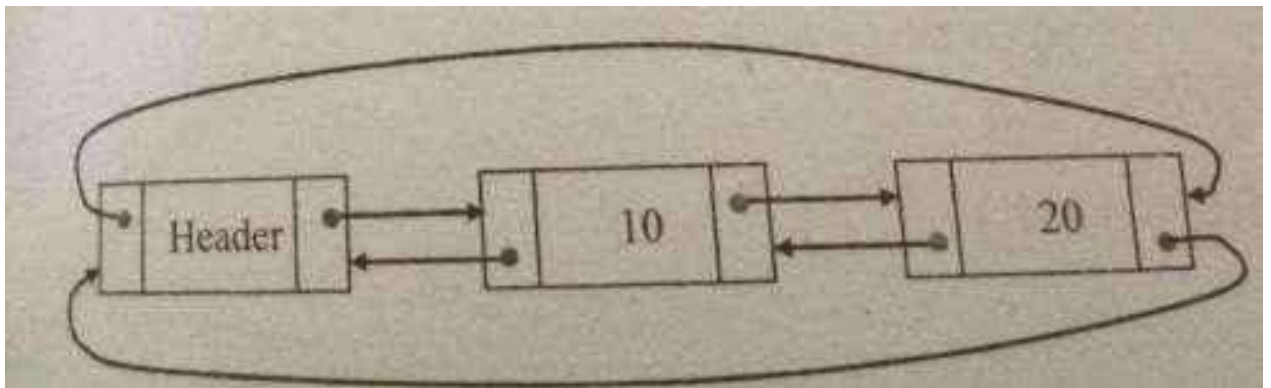
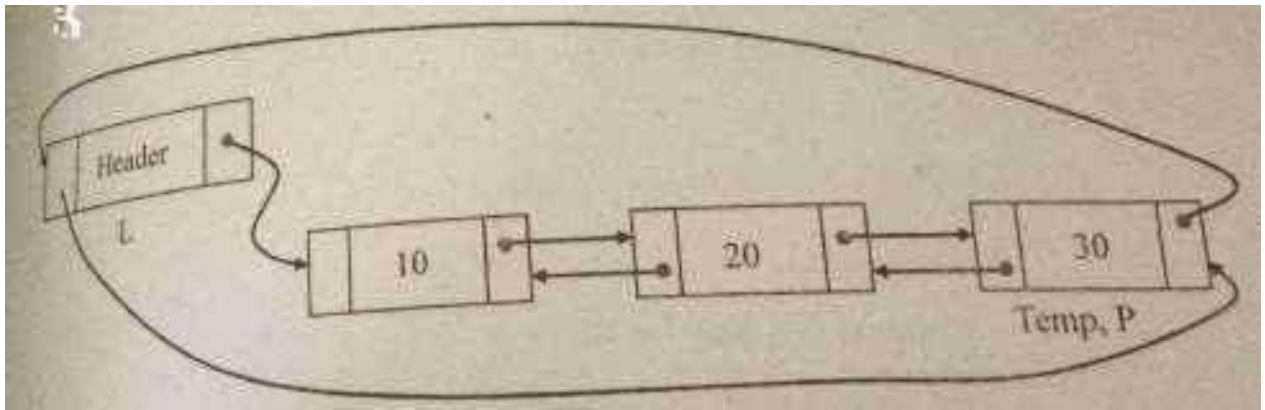
```
void del_mid(int X,List L)
{
    Position p, temp;
    p=FindPrevious(X);
    if(!IsLast(p,L))
    {
        temp=p->next;
        p->next=temp->next;
        temp->next->prev=p;
        free(temp);
    }
}
```

Delete (20, L)



Routine to delete an element at the last position

```
void del_last(List L)
{
    position p, temp;
    p=L;
    while(p->next!=L)
        p=p->next;
    temp=p;
    p->next->prev=L;
    L->prev=p->prev;
    free(temp);
}
```



Advantages of Circular linked List

- It allows to traverse the list starting at any point.
- It allows quick access to the first and last records.
- Circularly doubly linked list allows to traverse the list in either direction.

Applications of List:

1. Polynomial ADT
2. Radix sort
3. Multilist

Polynomial Manipulation

Polynomial manipulations such as addition, subtraction & differentiation etc.. can be performed using linked list

Declaration for Linked list implementation of Polynomial ADT

```
struct poly
{
int coeff;
int power;
struct poly *next;
}*list1,*list2,*list3;
```

Creation of the Polynomial

```
poly create(poly*head1,poly*newnode1)
{
    poly *ptr;
    if(head1==NULL)
    {
        head1=newnode1;
        return (head1);
    }
    else
    {
        ptr=head1;
        while(ptr->next!=NULL)
        ptr=ptr->next;
        ptr->next=newnode1;
    }
    return(head1);
}
```

Addition of two polynomials

```
void add()
{
    poly *ptr1, *ptr2, *newnode ;
    ptr1= list1;
    ptr2 = list2;
    while( ptr1 != NULL && ptr2 != NULL )
    {
        newnode = (struct poly*)malloc( sizeof ( struct poly ));
        if( ptr1 -> power == ptr2 -> power )
        {
            newnode -> coeff = ptr1 -> coeff + ptr2 -> coeff;
```

```

        newnode -> power = ptr1 -> power ;
newnode -> next = NULL;
list3 = create( list3, newnode );
ptr1 = ptr1 -> next;
ptr2 = ptr2 -> next;
    }
    else if(ptr1 -> power > ptr2 -> power )
    {
        newnode -> coeff = ptr1 -> coeff;
        newnode -> power = ptr1 -> power;
        newnode -> next = NULL;
        list3 = create( list3, newnode );
        ptr1 = ptr1 -> next;
    }
    else
    {
        newnode -> coeff = ptr2 -> coeff;
        newnode -> power = ptr2 -> power;
        newnode -> next = NULL;
        list3 = create( list3, newnode );
ptr2 = ptr2 -> next;
    }
}
}
}

```

Subtraction of two polynomial

```

{
void sub() poly *ptr1, *ptr2, *newnode;
    ptr1 = list1;
    ptr2 = list2;
    while( ptr1 != NULL && ptr2 != NULL )
    {
        newnode = (struct poly*)malloc( sizeof (struct poly)) ;
        if(ptr1->power==ptr2->power)
        {

```

```

        newnode->coeff=(ptr1->coeff)-(ptr2->coeff);
        newnode->power=ptr1->power;
newnode->next=NULL;
list3=create(list3,newnode);
ptr1=ptr1->next;

ptr2=ptr2->next;
    }
    else
    {
        if(ptr1->power>ptr2->power)
        {
            newnode->coeff=ptr1->coeff;
            newnode->power=ptr1->power;
            newnode->next=NULL;
            list3=create(list3,newnode);
            ptr1=ptr1->next;
        }
        else
        {
            newnode->coeff=-(ptr2->coeff);
            newnode->power=ptr2->power;
            newnode->next=NULL;
            list3=create(list3,newnode);
            ptr2=ptr2->next;
        }
    }
}

```

Polynomial Differentiation:

```

void diff()
{
    poly *ptr1, *newnode;
    ptr1 = list1;
    while( ptr1 != NULL)
    {
        newnode = (struct poly*)malloc( sizeof (struct poly));
        newnode->coeff=(ptr1->coeff)*(ptr1->power);
        newnode->power=ptr1->power-1;
    }
}

```

```

newnode->next=NULL;
list3=create(list3,newnode);
ptr1=ptr1->next;
}
}

```

Polynomial Multiplication

```

void mul()
{
    poly *ptr1, *ptr2, *newnode ;
    ptr1= list1;
    ptr2 = list2;
    while( ptr1 != NULL && ptr2 != NULL )
    {
        newnode = (struct poly*)malloc( sizeof ( struct poly ));
        if( ptr1 -> power == ptr2 -> power )
        {
            newnode -> coeff = ptr1 -> coeff * ptr2 -> coeff;
            newnode -> power = ptr1 -> power+ptr2->power; ;
            newnode -> next = NULL;
            list3 = create( list3, newnode );
            ptr1 = ptr1 -> next;
            ptr2 = ptr2 -> next;
        }
    }
}

```